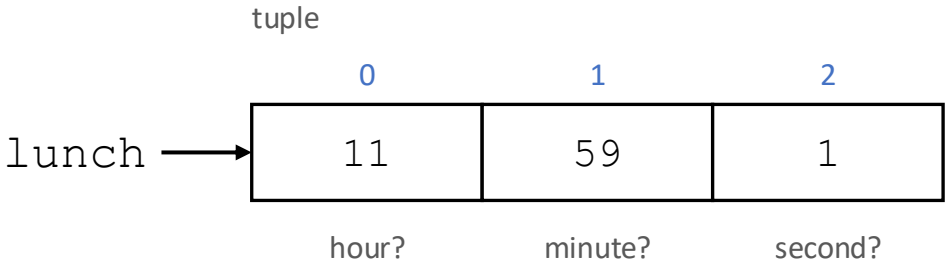
Classes and Objects

NUMBERS THAT BELONG TOGETHER

- A time of day has three parts: **hour**, **minute**, **second**
- From the tuples deck: a fixed group of things that belong together is a tuple
- That works, but which number is the minute?
- We can intuit that from context, but someone unfamiliar with our code might not immediately see it, since `lunch[1]` doesn't say what it means

```
lunch = (11, 59, 1)
print(lunch[1])
```

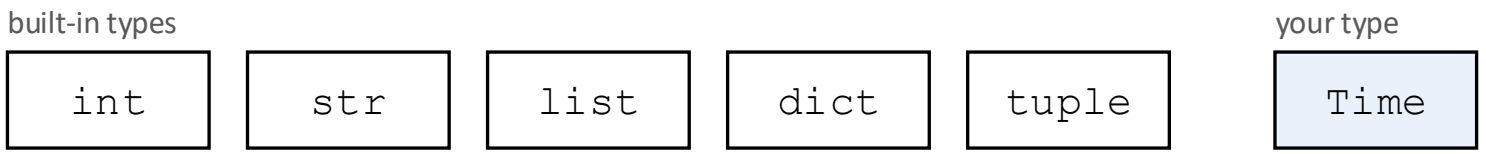
```
59
```



A CLASS IS A NEW TYPE

- Python comes with built-in types: `int`, `str`, `list`, `dict`, `tuple`
- A **class** is a type you define yourself
- `class`, a name, a colon, then an indented body
- Class names start with a capital letter
- For now the body is only a docstring

```
class Time:
    """Represents a time of day."""
```



MAKING AN OBJECT

- Call the class like a function to make a new object
- The new object is an **instance** of the class
- Making an object is called *instantiation*

```
lunch = Time()
print(type(lunch))
print(lunch)
```

Predict: what do the last two lines print?

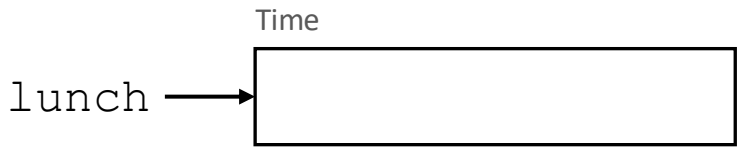
MAKING AN OBJECT

- Call the class like a function to make a new object
- The new object is an **instance** of the class
- Making an object is called *instantiation*

```
lunch = Time()
print(type(lunch))
print(lunch)
```

```
<class '__main__.Time'>
<__main__.Time object at 0x7f3a2c1b9e10>
```

Your address will be different

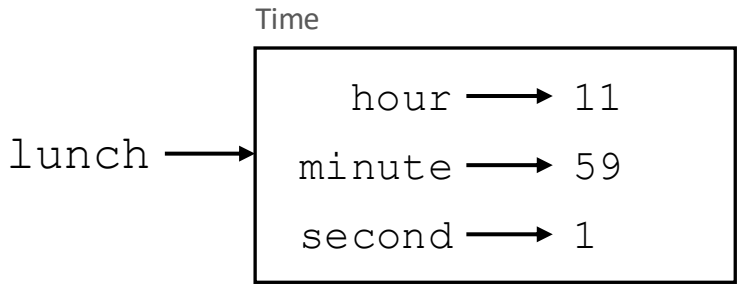


ATTRIBUTES

- An object can hold named values, called **attributes**
- Use a dot: `lunch.hour` is “lunch's hour”
- Assigning to an attribute creates it
- Reading one works like reading a variable

```
lunch.hour = 11
lunch.minute = 59
lunch.second = 1
print(lunch.minute)
```

```
59
```



NAMES INSTEAD OF POSITIONS

- Same three numbers, but now each one has a name
- The code is much closer to being self-documenting

tuple

```
lunch = (11, 59, 1)
print(lunch[1])
```

```
59
```

Time

```
lunch = Time()
lunch.hour = 11
lunch.minute = 59
lunch.second = 1
print(lunch.minute)
```

```
59
```

PASSING AN OBJECT TO A FUNCTION

- An object can be an argument, like any other value
- Inside, the function reads the object's attributes
- `.zfill(2)` pads a string with 0s until it is 2 characters long

```
def print_time(time):
    hh = str(time.hour).zfill(2)
    mm = str(time.minute).zfill(2)
    ss = str(time.second).zfill(2)
    print(hh + ':' + mm + ':' + ss)

print_time(lunch)
```

```
11:59:01
```

FUNCTIONS THAT MAKE OBJECTS

- `make_time` builds a `Time`, fills in its attributes, and returns it
- Now a time takes one line to make
- `time.hour = hour`: the left is an attribute, the right is the parameter

```
def make_time(hour, minute, second):
    time = Time()
    time.hour = hour
    time.minute = minute
    time.second = second
    return time

start = make_time(9, 20, 0)
print_time(start)
```

```
09:20:00
```

OBJECTS ARE MUTABLE

- Attributes can change after the object is made
- `increment_time` adds to a time's attributes, and returns nothing
- A movie starts at 9:20 and runs 1 hour, 32 minutes. When does it end?

```
def increment_time(time, hours, minutes, seconds):
    time.hour += hours
    time.minute += minutes
    time.second += seconds

start = make_time(9, 20, 0)
increment_time(start, 1, 32, 0)
print_time(start)
```

Predict: what does the last line print?

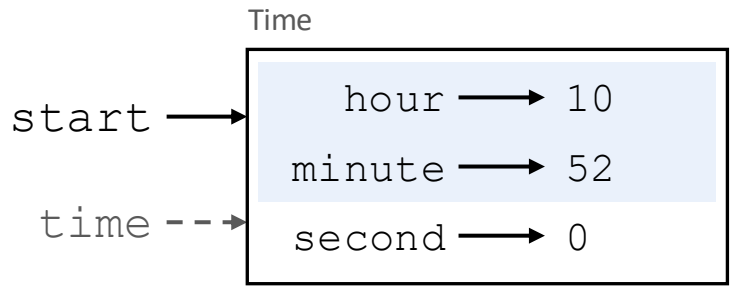
OBJECTS ARE MUTABLE

- Attributes can change after the object is made
- `increment_time` adds to a time's attributes, and returns nothing
- A movie starts at 9:20 and runs 1 hour, 32 minutes. When does it end?

```
def increment_time(time, hours, minutes, seconds):
    time.hour += hours
    time.minute += minutes
    time.second += seconds

start = make_time(9, 20, 0)
increment_time(start, 1, 32, 0)
print_time(start)
```

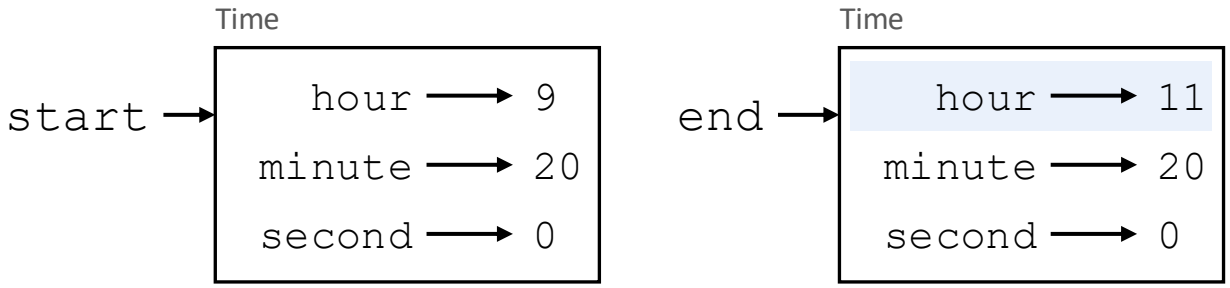
10:52:00



time is the parameter, while increment_time runs

COPYING AN OBJECT

- To change a time without changing the original, copy it first
- `copy` makes a new object with the same attributes
- `is` checks whether two variables refer to the same object




```
from copy import copy

start = make_time(9, 20, 0)
end = copy(start)
end.hour = 11
print_time(start)
print_time(end)
print(start is end)
```

```
09:20:00
11:20:00
False
```

PURE FUNCTIONS

- A **pure function** doesn't change its arguments; it returns a new value
- `add_time` copies, changes the copy, and returns it
- `increment_time` changes its argument; `add_time` leaves it alone

```
def add_time(time, hours, minutes, seconds):
    total = copy(time)
    increment_time(total, hours, minutes, seconds)
    return total
```

```
start = make_time(9, 20, 0)
end = add_time(start, 1, 32, 0)
print_time(end)
print_time(start)
```

```
10:52:00
09:20:00
```

QUESTIONS YOU MIGHT BE ASKING

- **Why not a dictionary?** It would work. A class gives the thing a type of its own, and later a class can carry its own functions
- **Is this a tuple with names?** Close, but you can change attributes. You can't change a tuple
- **Does add_time always work?** No: try a movie that starts at 9:40. The book chapter fixes this in 14.7 and 14.8

```
start = make_time(9, 40, 0)
end = add_time(start, 1, 32, 0)
print_time(end)
```

```
10:72:00
```

72 minutes isn't a time

WORDS TO KNOW

class	a type you define, like `Time`
instance	an object made from a class: `lunch = Time()`
attribute	a named value inside an object: `lunch.hour`
object diagram	a picture of an object and its attributes
pure function	only returns a value: doesn't change its arguments, print, or read input

Next: read Think Python Chapter 14. It goes further than these slides, especially 14.7 and 14.8